Ex2: Data Discovery and Preparation

Dataset: Telco Customer Churn (`Telco-Customer-Churn.csv`)

Upload `Telco-Customer-Churn.csv` to the Colab **Files** panel before running. If the file is not uploaded, the notebook downloads the same dataset automatically.

Task 1: Reuse Ex1 Steps 1 – 6 and 10

Step 1: Import the required libraries

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
pd.set_option('display.max_columns', None)
```

Step 2: Load the dataset – first 10 records, last 10 records, shape, column names

```
file_name = "Telco-Customer-Churn.csv"
if not os.path.exists(file_name):  # file not uploaded -> read the same dataset from GitHub
    file_name = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"

df = pd.read_csv(file_name)

print("First 10 records:")
display(df.head(10))
print("Last 10 records:")
display(df.tail(10))
print("Shape of the dataset:", df.shape)
print("Column names:", df.columns.tolist())
```

```
First 10 records:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL

	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod
0	No	Yes	No	No	No	No	Month-to-month	Yes	Electronic check
1	Yes	No	Yes	No	No	No	One year	No	Mailed check
2	Yes	Yes	No	No	No	No	Month-to-month	Yes	Mailed check
3	Yes	No	Yes	Yes	No	No	One year	No	Bank transfer (automatic)
4	No	No	No	No	No	No	Month-to-month	Yes	Electronic check
5	No	No	Yes	No	Yes	Yes	Month-to-month	Yes	Electronic check
6	No	Yes	No	No	Yes	No	Month-to-month	Yes	Credit card (automatic)
7	Yes	No	No	No	No	No	Month-to-month	No	Mailed check
8	No	No	Yes	Yes	Yes	Yes	Month-to-month	Yes	Electronic check

	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod
9	Yes	Yes	No	No	No	No	One year	No	Bank transfer (automatic)

	MonthlyCharges	TotalCharges	Churn
0	29.85	29.85	No
1	56.95	1889.5	No
2	53.85	108.15	Yes
3	42.30	1840.75	No
4	70.70	151.65	Yes
5	99.65	820.5	Yes
6	89.10	1949.4	No
7	29.75	301.9	No
8	104.80	3046.05	Yes
9	56.15	3487.95	No

Last 10 records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService
7033	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic
7035	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic
7036	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic

	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod
7033	No	No	No	No	No	No	Month-to-month	Yes	Credit card (automatic)
7034	Yes	Yes	Yes	No	Yes	No	Month-to-month	Yes	Credit card (automatic)
7035	No	No	No	No	Yes	No	Month-to-month	Yes	Bank transfer (automatic)
7036	No	Yes	Yes	Yes	Yes	Yes	One year	No	Electronic check
7037	No internet service	No internet service	No internet service	No internet service	No internet service	No internet service	Two year	Yes	Bank transfer (automatic)
7038	Yes	No	Yes	Yes	Yes	Yes	One year	Yes	Mailed check
7039	No	Yes	Yes	No	Yes	Yes	One year	Yes	Credit card (automatic)
7040	Yes	No	No	No	No	No	Month-to-month	Yes	Electronic check
7041	No	No	No	No	No	No	Month-to-month	Yes	Mailed check
7042	Yes	No	Yes	Yes	Yes	Yes	Two year	Yes	Bank transfer (automatic)

	MonthlyCharges	TotalCharges	Churn
7033	69.50	2625.25	No
7034	102.95	6886.25	Yes
7035	78.70	1495.1	No
7036	60.65	743.3	No
7037	21.15	1419.4	No
7038	84.80	1990.5	No
7039	103.20	7362.9	No
7040	29.60	346.45	No
7041	74.40	306.6	Yes
7042	105.65	6844.5	No

```
Shape of the dataset: (7043, 21)
Column names: ['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn']
```

Step 3: Explore the dataset – information, summary statistics, missing values, duplicates

```
df.info()
display(df.describe().T)
display(df.describe(exclude='number').T)

# TotalCharges is stored as text with blank strings -> convert to numeric (blanks become NaN)
print("Blank strings in TotalCharges:", (df['TotalCharges'].astype(str).str.strip() == '').sum())
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

print("\nMissing values in each column:")
print(df.isnull().sum())
print("Number of duplicate records:", df.duplicated().sum())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
 17  PaymentMethod     7043 non-null   str
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str
 20  Churn             7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

	count	mean	std	min	25%	50%	75%	max
SeniorCitizen	7043.0	0.162147	0.368612	0.00	0.0	0.00	0.00	1.00
tenure	7043.0	32.371149	24.559481	0.00	9.0	29.00	55.00	72.00
MonthlyCharges	7043.0	64.761692	30.090047	18.25	35.5	70.35	89.85	118.75

	count	unique	top	freq
customerID	7043	7043	7590-VHVEG	1
gender	7043	2	Male	3555
Partner	7043	2	No	3641
Dependents	7043	2	No	4933
PhoneService	7043	2	Yes	6361
MultipleLines	7043	3	No	3390
InternetService	7043	3	Fiber optic	3096
OnlineSecurity	7043	3	No	3498
OnlineBackup	7043	3	No	3088
DeviceProtection	7043	3	No	3095
TechSupport	7043	3	No	3473
StreamingTV	7043	3	No	2810
StreamingMovies	7043	3	No	2785
Contract	7043	3	Month-to-month	3875
PaperlessBilling	7043	2	Yes	4171
PaymentMethod	7043	4	Electronic check	2365
TotalCharges	7043	6531	20.2	11
Churn	7043	2	No	5174

```
Blank strings in TotalCharges: 11

Missing values in each column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
```

```
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges        11
Churn                0
dtype: int64
Number of duplicate records: 0
```

Step 4: Clean the data – remove duplicates, fill missing values (median / mode), verify

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset='customerID', keep='first')

num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(exclude='number').columns
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Shape after cleaning:", df.shape)
print("Total missing values after cleaning:", df.isnull().sum().sum())
```

```
Shape after cleaning: (7043, 21)
Total missing values after cleaning: 0
```

Step 5: Encode categorical columns (Label Encoding for binary, One-Hot Encoding for multi-category)

```
df_clean = df.copy()      # cleaned data with original labels (used for EDA)
df_encoded = df.copy()

binary_cols = ['gender', 'Partner', 'Dependents', 'Churn']
le = LabelEncoder()
for col in binary_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])
    print(f"{col}: {dict((c, i) for i, c in enumerate(le.classes_))}")

multi_cols = ['Contract', 'PaymentMethod', 'InternetService']
df_encoded = pd.get_dummies(df_encoded, columns=multi_cols, dtype=int)
print("Shape after encoding:", df_encoded.shape)
```

```
gender: {'Female': 0, 'Male': 1}
Partner: {'No': 0, 'Yes': 1}
Dependents: {'No': 0, 'Yes': 1}
Churn: {'No': 0, 'Yes': 1}
Shape after encoding: (7043, 28)
```

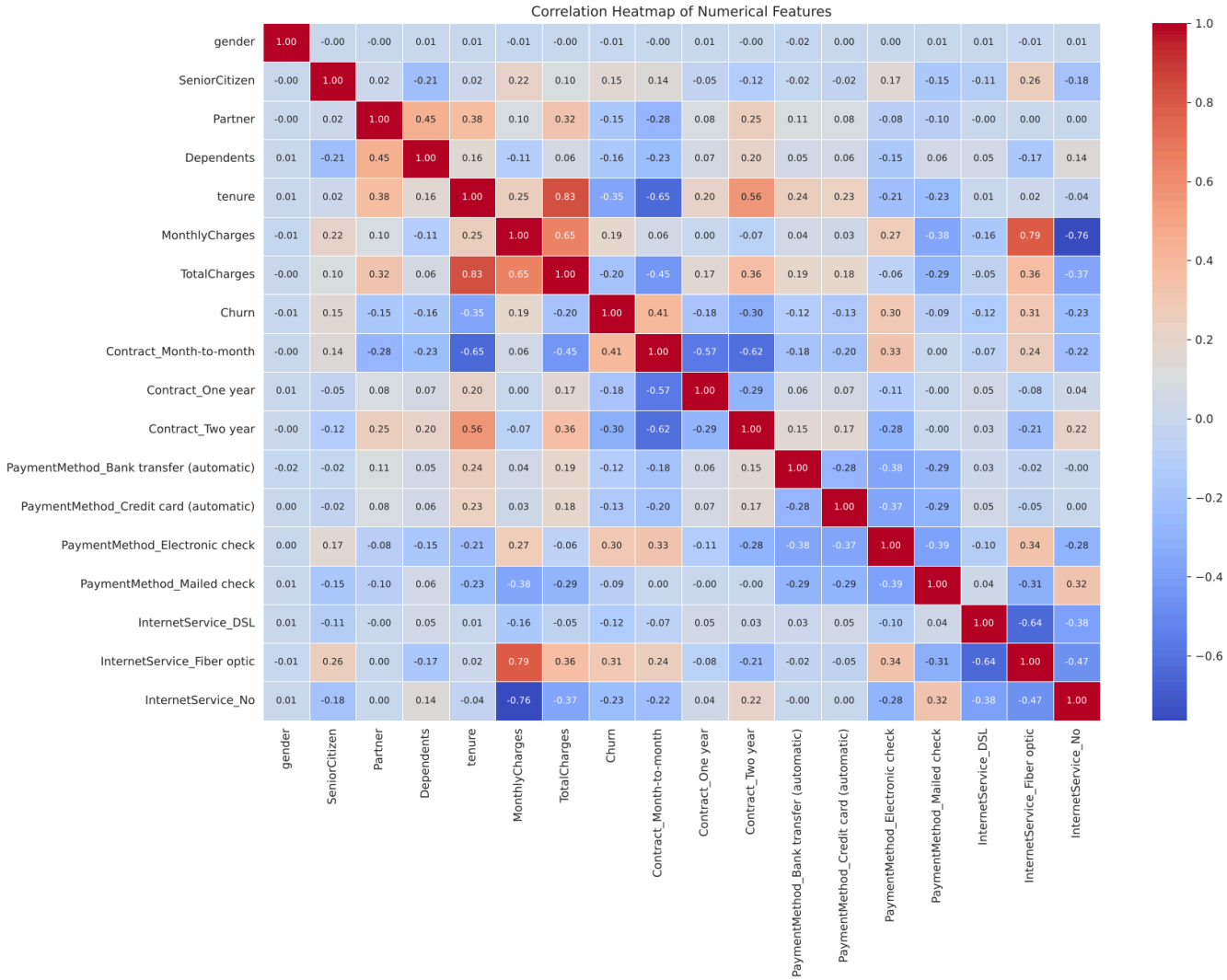
Step 6: Standardise Tenure, MonthlyCharges, TotalCharges

```
scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
scaler = StandardScaler()
df_encoded[scale_cols] = scaler.fit_transform(df_encoded[scale_cols])
display(df_encoded[scale_cols].describe().round(2))
```

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	-0.00	-0.00	-0.00
std	1.00	1.00	1.00
min	-1.32	-1.55	-1.00
25%	-0.95	-0.97	-0.83
50%	-0.14	0.19	-0.39
75%	0.92	0.83	0.66
max	1.61	1.79	2.83

Step 10: Correlation Heatmap of all numerical features

```
corr = df_encoded.select_dtypes(include='number').corr()
plt.figure(figsize=(16, 12))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5, annot_kws={'size': 8})
plt.title('Correlation Heatmap of Numerical Features')
plt.tight_layout()
plt.show()
```



Task 2: Distribution of Customer Churn, Gender, Contract Type, Payment Method

```
dist_cols = ['Churn', 'gender', 'Contract', 'PaymentMethod']
for col in dist_cols:
    counts = df_clean[col].value_counts()
    percent = df_clean[col].value_counts(normalize=True).mul(100).round(2)
    print(f"\n----- {col} -----")
    display(pd.DataFrame({'Count': counts, 'Percentage (%)': percent}))

fig, axes = plt.subplots(2, 2, figsize=(15, 10))
for ax, col in zip(axes.flat, dist_cols):
    order = df_clean[col].value_counts().index
    sns.countplot(data=df_clean, x=col, order=order, hue=col, palette='Set2', legend=False, ax=ax)
    for container in ax.containers:
        ax.bar_label(container)
    ax.set_title(f'Distribution of {col}')
    ax.tick_params(axis='x', rotation=15)
plt.tight_layout()
plt.show()
```

```
----- Churn -----
```

	Count	Percentage (%)
Churn		
No	5174	73.46
Yes	1869	26.54

```
----- gender -----
```

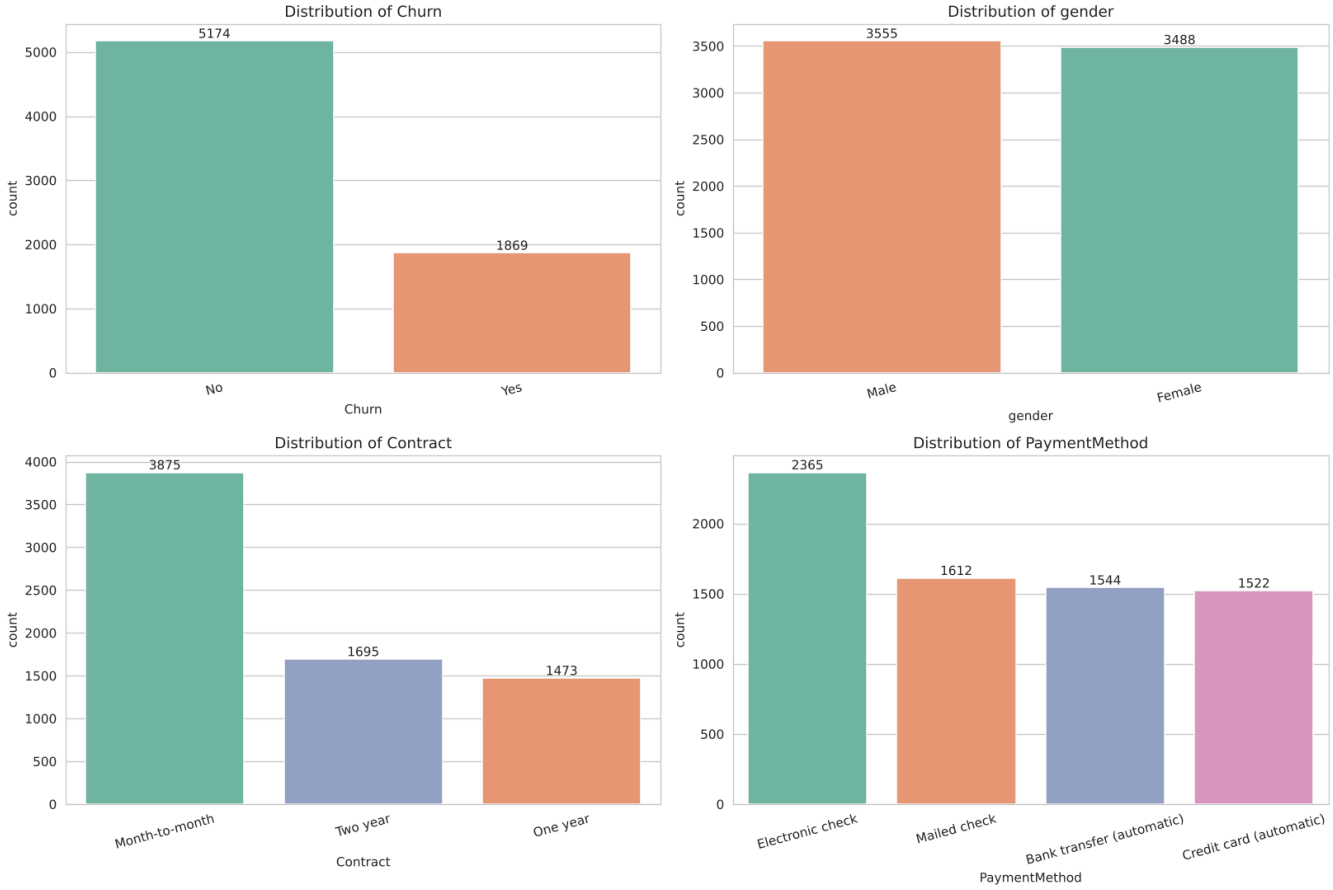
	Count	Percentage (%)
gender		
Male	3555	50.48
Female	3488	49.52

```
----- Contract -----
```

	Count	Percentage (%)
Contract		
Month-to-month	3875	55.02
Two year	1695	24.07
One year	1473	20.91

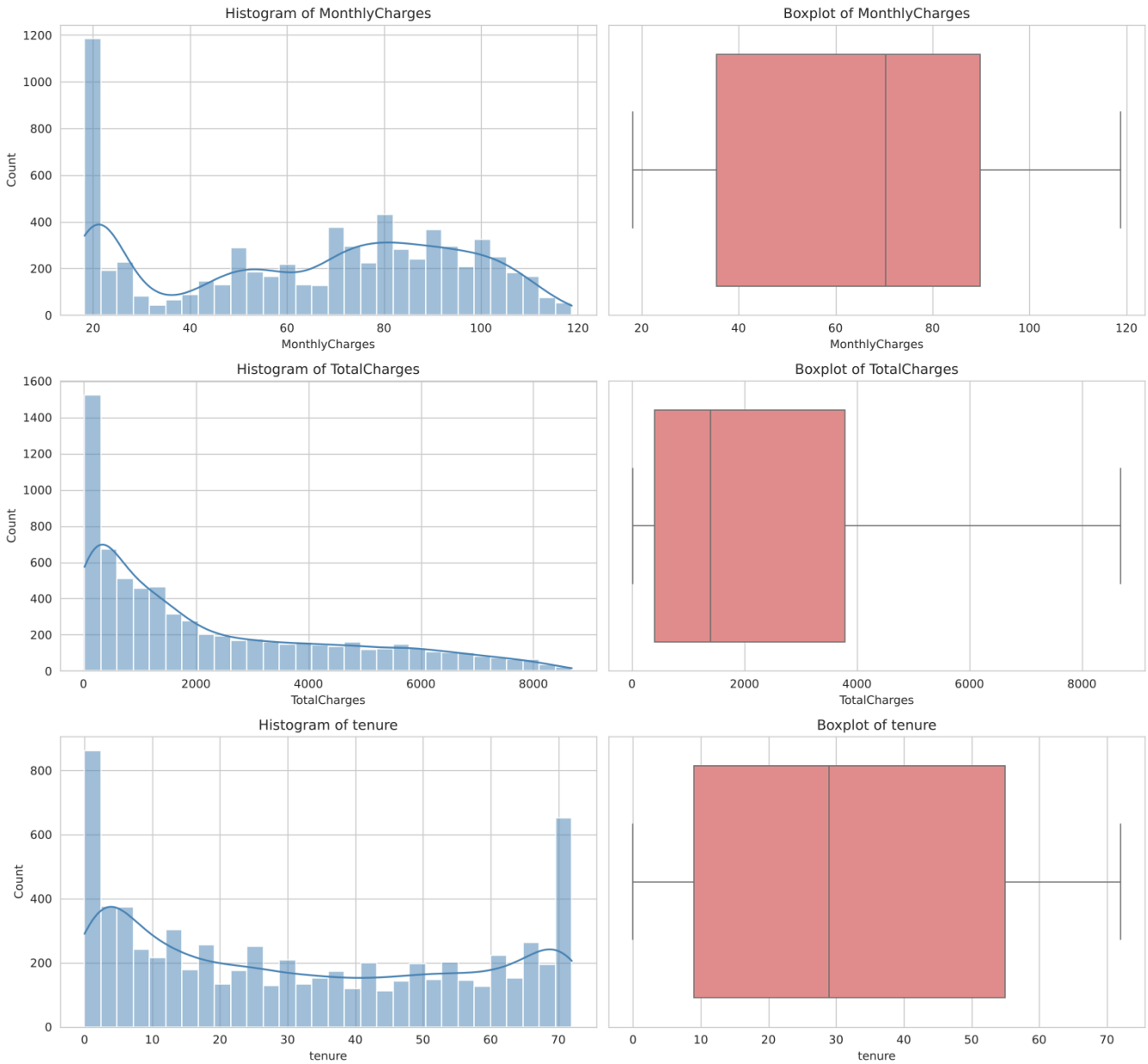
```
----- PaymentMethod -----
```

	Count	Percentage (%)
PaymentMethod		
Electronic check	2365	33.58
Mailed check	1612	22.89
Bank transfer (automatic)	1544	21.92
Credit card (automatic)	1522	21.61



Task 3: Histograms and Boxplots of MonthlyCharges, TotalCharges, Tenure

```
num_features = ['MonthlyCharges', 'TotalCharges', 'tenure']
fig, axes = plt.subplots(3, 2, figsize=(14, 13))
for i, col in enumerate(num_features):
    sns.histplot(df_clean[col], bins=30, kde=True, color='steelblue', ax=axes[i, 0])
    axes[i, 0].set_title(f'Histogram of {col}')
    sns.boxplot(x=df_clean[col], color='lightcoral', ax=axes[i, 1])
    axes[i, 1].set_title(f'Boxplot of {col}')
plt.tight_layout()
plt.show()
```



Task 4: Detect outliers using the Interquartile Range (IQR) method

```
outlier_summary = []
for col in num_features:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_out = ((df_clean[col] < lower) | (df_clean[col] > upper)).sum()
    outlier_summary.append([col, Q1, Q3, IQR, lower, upper, n_out])

outlier_df = pd.DataFrame(outlier_summary, columns=['Feature', 'Q1', 'Q3', 'IQR',
                                                    'Lower Bound', 'Upper Bound', 'No. of Outliers'])
display(outlier_df.round(2))
```

	Feature	Q1	Q3	IQR	Lower Bound	Upper Bound	No. of Outliers
0	MonthlyCharges	35.50	89.85	54.35	-46.02	171.38	0
1	TotalCharges	402.22	3786.60	3384.38	-4674.34	8863.16	0
2	tenure	9.00	55.00	46.00	-60.00	124.00	0

Task 5: Relationship of Contract Type, Internet Service, Monthly Charges and Tenure with Churn

```
fig, axes = plt.subplots(2, 2, figsize=(15, 11))
```

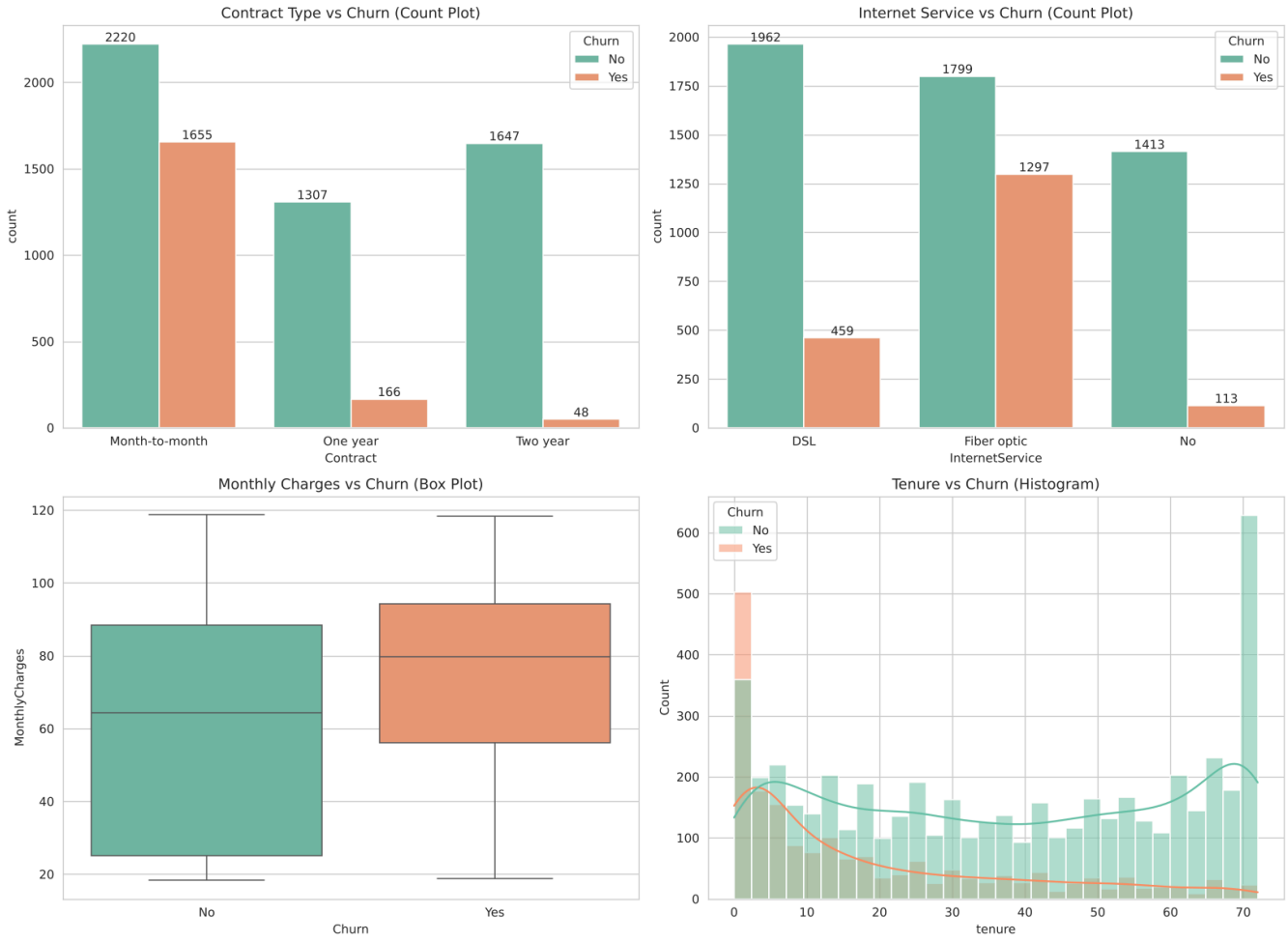
```
sns.countplot(data=df_clean, x='Contract', hue='Churn', palette='Set2', ax=axes[0, 0])
axes[0, 0].set_title('Contract Type vs Churn (Count Plot)')

sns.countplot(data=df_clean, x='InternetService', hue='Churn', palette='Set2', ax=axes[0, 1])
axes[0, 1].set_title('Internet Service vs Churn (Count Plot)')

sns.boxplot(data=df_clean, x='Churn', y='MonthlyCharges', hue='Churn', palette='Set2', legend=False, ax=axes[1, 0])
axes[1, 0].set_title('Monthly Charges vs Churn (Box Plot)')

sns.histplot(data=df_clean, x='tenure', hue='Churn', bins=30, kde=True, palette='Set2', ax=axes[1, 1])
axes[1, 1].set_title('Tenure vs Churn (Histogram)')

for ax in axes[0]:
    for container in ax.containers:
        ax.bar_label(container)
plt.tight_layout()
plt.show()
```



Task 6: Analytical Questions

```
def churn_rate(col):
    """Churn percentage for each category of a column."""
    return (df_clean.groupby(col)['Churn']
            .apply(lambda s: (s == 'Yes').mean() * 100)
            .round(2).sort_values(ascending=False))

churned = df_clean[df_clean['Churn'] == 'Yes']
```

Q1. Which contract type has the highest customer churn?

```
contract_churn = churn_rate('Contract')
display(contract_churn.to_frame('Churn Rate (%)'))
print(f"Answer: '{contract_churn.idxmax()}' contract has the highest churn ({contract_churn.max()}%).")
```

Contract	Churn Rate (%)
Month-to-month	42.71
One year	11.27
Two year	2.83

```
Answer: 'Month-to-month' contract has the highest churn (42.71%).
```

Q2. Which payment method is most frequently used by customers who churn?

```
pay_churned = churned['PaymentMethod'].value_counts()
display(pay_churned.to_frame('Churned Customers'))
print(f"Answer: '{pay_churned.idxmax()}' is used by {pay_churned.max()} of {len(churned)} churned customers "
      f"({pay_churned.max() / len(churned) * 100:.2f}%).")
```

PaymentMethod	Churned Customers
Electronic check	1071
Mailed check	308
Bank transfer (automatic)	258
Credit card (automatic)	232

```
Answer: 'Electronic check' is used by 1071 of 1869 churned customers (57.30%).
```

Q3. Do customers with shorter tenure tend to churn more often?

```
print("Average tenure (months):")
display(df_clean.groupby('Churn')['tenure'].mean().round(2).to_frame('Mean Tenure'))

df_clean['TenureGroup'] = pd.cut(df_clean['tenure'], bins=[-1, 12, 24, 48, 72],
                                 labels=['0-12', '13-24', '25-48', '49-72'])
tenure_churn = churn_rate('TenureGroup').sort_index()
display(tenure_churn.to_frame('Churn Rate (%)'))

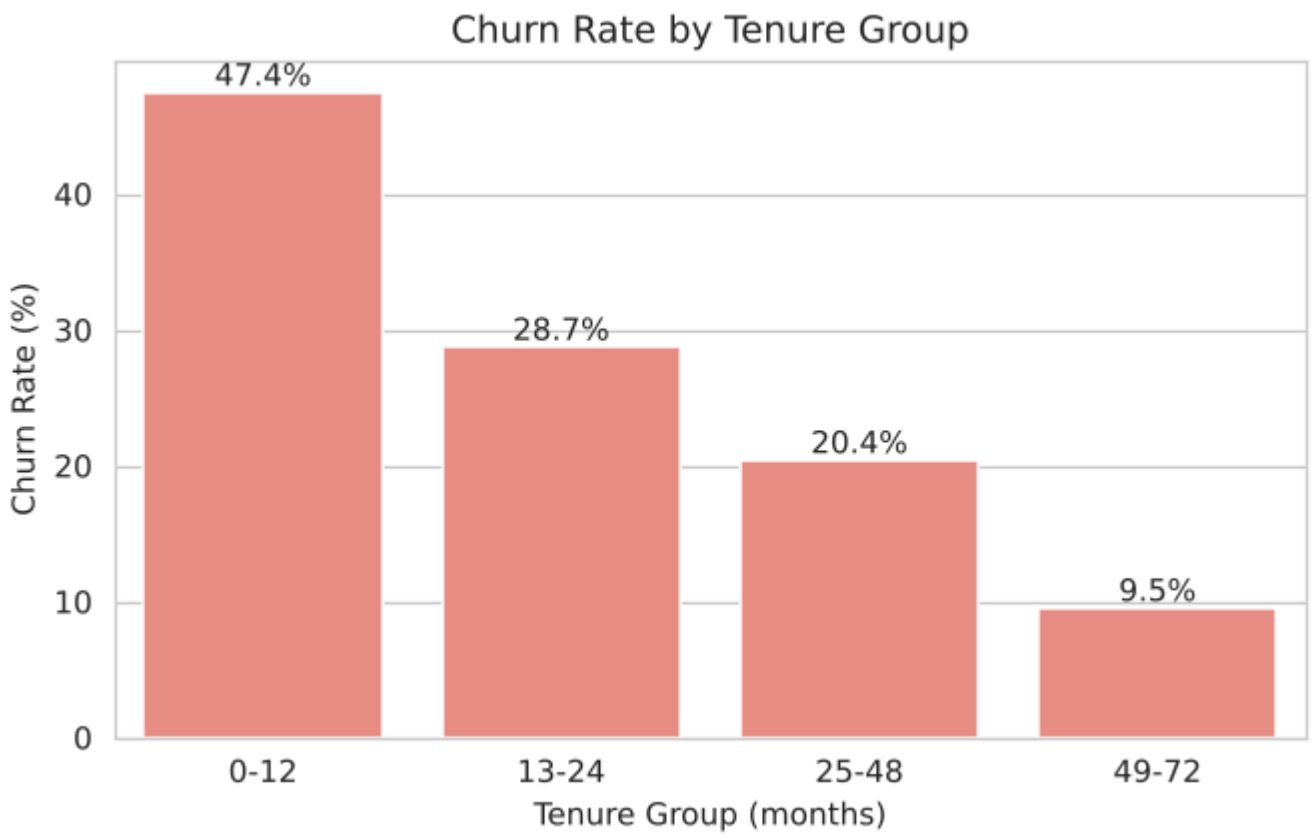
plt.figure(figsize=(7, 4))
ax = sns.barplot(x=tenure_churn.index.astype(str), y=tenure_churn.values, color='salmon')
ax.bar_label(ax.containers[0], fmt='%.1f%%')
plt.title('Churn Rate by Tenure Group')
plt.xlabel('Tenure Group (months)')
plt.ylabel('Churn Rate (%)')
plt.show()
df_clean = df_clean.drop(columns='TenureGroup')

print("Answer: Yes. Customers in their first 12 months churn the most, and churn falls steadily as tenure increases.")
```

```
Average tenure (months):
```

Churn	Mean Tenure
No	37.57
Yes	17.98

TenureGroup	Churn Rate (%)
0-12	47.44
13-24	28.71
25-48	20.39
49-72	9.51



Answer: Yes. Customers in their first 12 months churn the most, and churn falls steadily as tenure increases.

Q4. Which customer group has the highest monthly charges?

```
group_cols = ['Contract', 'InternetService', 'PaymentMethod', 'SeniorCitizen', 'Partner',
              'Dependents', 'gender', 'Churn']
rows = []
for col in group_cols:
    means = df_clean.groupby(col)['MonthlyCharges'].mean()
    rows.append([col, means.idxmax(), round(means.max(), 2)])
group_df = (pd.DataFrame(rows, columns=['Grouping', 'Group with Highest Charges', 'Avg MonthlyCharges ($)'])
            .sort_values('Avg MonthlyCharges ($)', ascending=False))
display(group_df)
top = group_df.iloc[0]
print(f"Answer: Customers with {top['Grouping']} = '{top['Group with Highest Charges']}' "
      f"have the highest average monthly charges (${top['Avg MonthlyCharges ($)']}).")
```

	Grouping	Group with Highest Charges	Avg MonthlyCharges ($)
1	InternetService	Fiber optic	91.50
3	SeniorCitizen	1	79.82
2	PaymentMethod	Electronic check	76.26
7	Churn	Yes	74.44
4	Partner	Yes	67.78
5	Dependents	No	67.00
0	Contract	Month-to-month	66.40
6	gender	Female	65.20

Answer: Customers with InternetService = 'Fiber optic' have the highest average monthly charges ($91.5).

Q5. Which Internet Service category experiences the highest churn?

```
internet_churn = churn_rate('InternetService')
display(internet_churn.to_frame('Churn Rate (%)'))
print(f"Answer: '{internet_churn.idxmax()}' customers have the highest churn ({internet_churn.max()}%).")
```

	Churn Rate (%)
InternetService	
Fiber optic	41.89
DSL	18.96
No	7.40

Answer: 'Fiber optic' customers have the highest churn (41.89%).

Q6. Is there any relationship between monthly charges and churn?

```
display(df_clean.groupby('Churn')['MonthlyCharges'].describe().round(2))

churn_flag = (df_clean['Churn'] == 'Yes').astype(int)
corr_mc = df_clean['MonthlyCharges'].corr(churn_flag)
print(f"Correlation between MonthlyCharges and Churn: {corr_mc:.3f}")

charge_band = pd.qcut(df_clean['MonthlyCharges'], q=4, labels=['Low', 'Medium', 'High', 'Very High'])
band_churn = churn_flag.groupby(charge_band).mean().mul(100).round(2)
display(band_churn.to_frame('Churn Rate (%)'))
print("Answer: Yes, a positive but weak relationship. Churned customers pay more per month on average, "
      "and the churn rate is much higher in the High / Very High charge bands than in the Low band.")
```

	count	mean	std	min	25%	50%	75%	max
Churn								
No	5174.0	61.27	31.09	18.25	25.10	64.43	88.4	118.75
Yes	1869.0	74.44	24.67	18.85	56.15	79.65	94.2	118.35

```
Correlation between MonthlyCharges and Churn: 0.193
```

	Churn Rate (%)
MonthlyCharges	
Low	11.24
Medium	24.58
High	37.51
Very High	32.88

```
Answer: Yes, a positive but weak relationship. Churned customers pay more per month on average, and the churn rate is much higher in
the High / Very High charge bands than in the Low band.
```

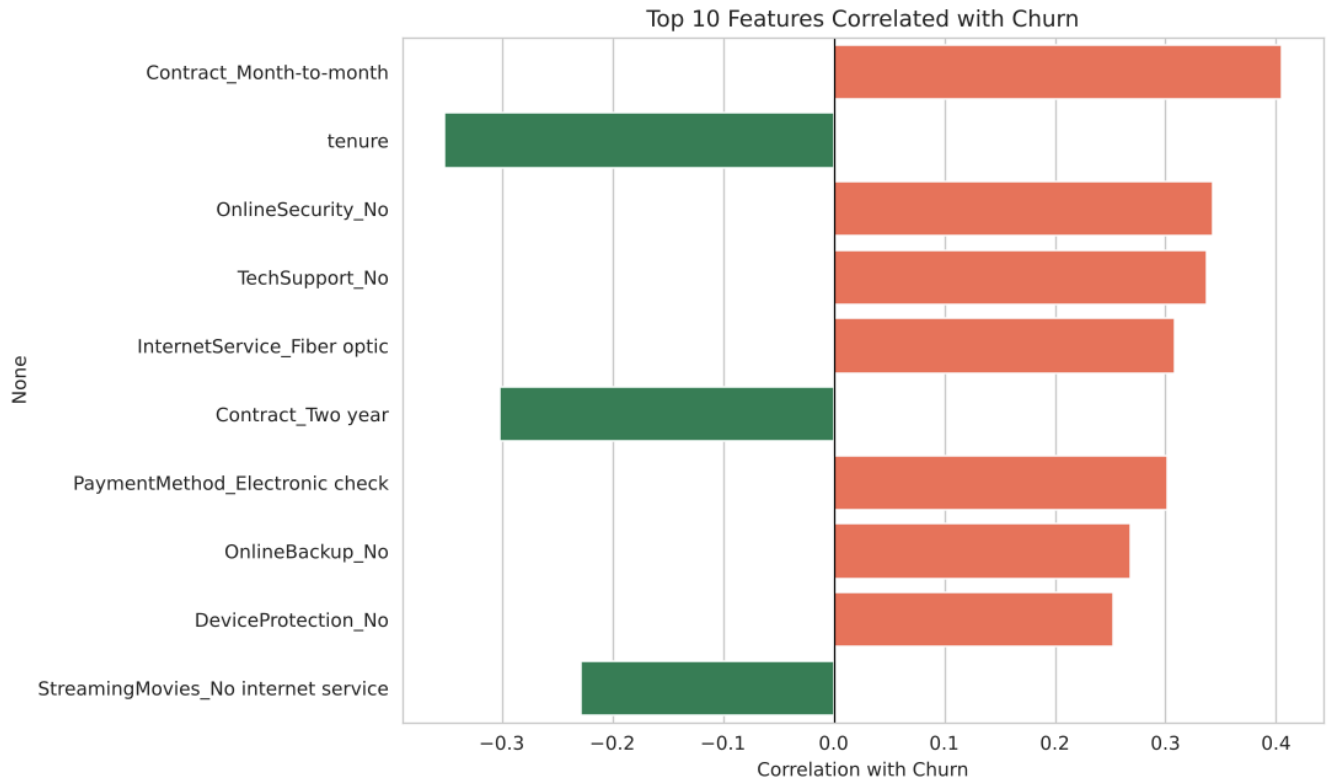
Q7. Which features appear to be the strongest indicators of customer churn?

```
full_encoded = pd.get_dummies(df_clean.drop(columns=['customerID']), dtype=int)
churn_corr = (full_encoded.corr()['Churn_Yes']
              .drop(['Churn_Yes', 'Churn_No'])
              .sort_values(key=abs, ascending=False))
top10 = churn_corr.head(10)
display(top10.round(3).to_frame('Correlation with Churn'))

plt.figure(figsize=(10, 6))
sns.barplot(x=top10.values, y=top10.index, hue=top10.values > 0,
            palette={True: 'tomato', False: 'seagreen'}, legend=False)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Top 10 Features Correlated with Churn')
plt.xlabel('Correlation with Churn')
plt.tight_layout()
plt.show()

print("Answer: The strongest indicators are a month-to-month contract, short tenure, no online security, "
      "no tech support, fiber optic internet and paying by electronic check (positive = more churn, negative = less churn).")
```

	Correlation with Churn
Contract_Month-to-month	0.405
tenure	-0.352
OnlineSecurity_No	0.343
TechSupport_No	0.337
InternetService_Fiber optic	0.308
Contract_Two year	-0.302
PaymentMethod_Electronic check	0.302
OnlineBackup_No	0.268
DeviceProtection_No	0.252
StreamingMovies_No internet service	-0.228



Answer: The strongest indicators are a month-to-month contract, short tenure, no online security, no tech support, fiber optic internet and paying by electronic check (positive = more churn, negative = less churn).

Task 7: Split the cleaned dataset into Training (80%) and Testing (20%)

Before splitting, the dataset is made fully numeric for modelling: the ID column is dropped and the remaining categorical columns are encoded in the same way (Label Encoding for 2 categories, One-Hot Encoding for more).

```
df_final = df_encoded.drop(columns=['customerID'])

remaining_cat = df_final.select_dtypes(exclude='number').columns.tolist()
print("Remaining categorical columns:", remaining_cat)

for col in remaining_cat:
    if df_final[col].nunique() == 2:
        df_final[col] = LabelEncoder().fit_transform(df_final[col])
df_final = pd.get_dummies(df_final, columns=df_final.select_dtypes(exclude='number').columns.tolist(), dtype=int)

print("Final dataset shape:", df_final.shape)
display(df_final.head())
```

```
Remaining categorical columns: ['PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
'TechSupport', 'StreamingTV', 'StreamingMovies', 'PaperlessBilling']
Final dataset shape: (7043, 41)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	PaperlessBilling	MonthlyCharges	TotalCharges
0	0	0	1	0	-1.277445	0	1	-1.160323	-0.994242
1	1	0	0	0	0.066327	1	0	-0.259629	-0.173244
2	1	0	0	0	-1.236724	1	1	-0.362660	-0.959674
3	1	0	0	0	0.514251	0	0	-0.746535	-0.194766
4	0	0	0	0	-1.236724	1	1	0.197365	-0.940470

	Churn	Contract_Month-to-month	Contract_One year	Contract_Two year	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check	PaymentMethod_Mailed check	InternetService_DSL
0	0	1	0	0	0	0	1	0	1
1	0	0	1	0	0	0	0	1	1
2	1	1	0	0	0	0	0	1	1
3	0	0	1	0	1	0	0	0	1
4	1	1	0	0	0	0	1	0	0

	InternetService_Fiber optic	InternetService_No	MultipleLines_No	MultipleLines_No phone service	MultipleLines_Yes	OnlineSecurity_No	OnlineSecurity_No internet service	OnlineSecurity_Yes	OnlineBackup_No
0	0	0	0	1	0	1	0	0	0
1	0	0	1	0	0	0	0	1	1

	InternetService_Fiber optic	InternetService_No	MultipleLines_No	MultipleLines_No phone service	MultipleLines_Yes	OnlineSecurity_No	OnlineSecurity_No internet service	OnlineSecurity_Yes	OnlineBackup_No
2	0	0	1	0	0	0	0	1	0
3	0	0	0	1	0	0	0	1	1
4	1	0	1	0	0	1	0	0	1

	OnlineBackup_No internet service	OnlineBackup_Yes	DeviceProtection_No	DeviceProtection_No internet service	DeviceProtection_Yes	TechSupport_No	TechSupport_No internet service	TechSupport_Yes	StreamingTV_No
0	0	1	1	0	0	1	0	0	1
1	0	0	0	0	1	1	0	0	1
2	0	1	1	0	0	1	0	0	1
3	0	0	0	0	1	0	0	1	1
4	0	0	1	0	0	1	0	0	1

	StreamingTV_No internet service	StreamingTV_Yes	StreamingMovies_No	StreamingMovies_No internet service	StreamingMovies_Yes
0	0	0	1	0	0
1	0	0	1	0	0
2	0	0	1	0	0
3	0	0	1	0	0
4	0	0	1	0	0

```
X = df_final.drop(columns=['Churn'])
y = df_final['Churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print("X_train:", X_train.shape, " y_train:", y_train.shape)
print("X_test :", X_test.shape, " y_test :", y_test.shape)
print("\nChurn distribution in training set (%):")
print(y_train.value_counts(normalize=True).mul(100).round(2))
print("\nChurn distribution in testing set (%):")
print(y_test.value_counts(normalize=True).mul(100).round(2))
```

```
X_train: (5634, 40)  y_train: (5634,)
X_test : (1409, 40)  y_test : (1409,)

Churn distribution in training set (%):
Churn
0    73.46
1    26.54
Name: proportion, dtype: float64

Churn distribution in testing set (%):
Churn
0    73.46
1    26.54
Name: proportion, dtype: float64
```

Task 8: Save the final processed dataset as `CustomerChurn_Preprocessed.csv`

```
df_final.to_csv('CustomerChurn_Preprocessed.csv', index=False)
print("Saved CustomerChurn_Preprocessed.csv with shape", df_final.shape)
display(pd.read_csv('CustomerChurn_Preprocessed.csv').head())
```

```
Saved CustomerChurn_Preprocessed.csv with shape (7043, 41)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	PaperlessBilling	MonthlyCharges	TotalCharges
0	0	0	1	0	-1.277445	0	1	-1.160323	-0.994242
1	1	0	0	0	0.066327	1	0	-0.259629	-0.173244
2	1	0	0	0	-1.236724	1	1	-0.362660	-0.959674
3	1	0	0	0	0.514251	0	0	-0.746535	-0.194766
4	0	0	0	0	-1.236724	1	1	0.197365	-0.940470

	Churn	Contract_Month-to-month	Contract_One year	Contract_Two year	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check	PaymentMethod_Mailed check	InternetService_DSL
0	0	1	0	0	0	0	1	0	1
1	0	0	1	0	0	0	0	1	1
2	1	1	0	0	0	0	0	1	1
3	0	0	1	0	1	0	0	0	1
4	1	1	0	0	0	0	1	0	0

	InternetService_Fiber optic	InternetService_No	MultipleLines_No	MultipleLines_No phone service	MultipleLines_Yes	OnlineSecurity_No	OnlineSecurity_No internet service	OnlineSecurity_Yes	OnlineBackup_No
0	0	0	0	1	0	1	0	0	0

	InternetService_Fiber optic	InternetService_No	MultipleLines_No	MultipleLines_No phone service	MultipleLines_Yes	OnlineSecurity_No	OnlineSecurity_No internet service	OnlineSecurity_Yes	OnlineBackup_No
1	0	0	1	0	0	0	0	1	1
2	0	0	1	0	0	0	0	1	0
3	0	0	0	1	0	0	0	1	1
4	1	0	1	0	0	1	0	0	1

	OnlineBackup_No internet service	OnlineBackup_Yes	DeviceProtection_No	DeviceProtection_No internet service	DeviceProtection_Yes	TechSupport_No	TechSupport_No internet service	TechSupport_Yes	StreamingTV_No
0	0	1	1	0	0	1	0	0	1
1	0	0	0	0	1	1	0	0	1
2	0	1	1	0	0	1	0	0	1
3	0	0	0	0	1	0	0	1	1
4	0	0	1	0	0	1	0	0	1

	StreamingTV_No internet service	StreamingTV_Yes	StreamingMovies_No	StreamingMovies_No internet service	StreamingMovies_Yes
0	0	0	1	0	0
1	0	0	1	0	0
2	0	0	1	0	0
3	0	0	1	0	0
4	0	0	1	0	0